Relatório Técnico - Crédito 2

Implementação do FCM e Obtenção Empírica da Função Tempo

Equipe 8 - Memory Flattening (Achatamento de Memória 1D)
Membro 1: Samuel Cesar
Membro 2: Pedro Henrique
Membro 3: Pedro Lucas
Universidade do Estado da Bahia (UNEB) - DCET I

5 de novembro de 2026

1 Introdução e Escopo

Este documento apresenta os resultados do Crédito 2 da disciplina de Engenharia de Programas. O objetivo é descrever a implementação desenvolvida *from scratch* do algoritmo **Fuzzy C-Means (FCM)**, respeitando rigorosamente a restrição arquitetural sorteada pela equipe, e apresentar o levantamento do Tempo Empírico (T_{exp}) em função do número de pixels (n) e clusters (K).

2 Análise da Restrição Arquitetural

Restrição Sorteada: Linearização da Matriz de Pertinência U em 1D"(Memory Flattening)

Seguindo às diretrizes da **Ação Cirúrgica** e **Preservação de Escopo** do código base (`FCM_PURO_v3.py`), a intervenção arquitetural restringiu-se exclusivamente à matriz de pertinência U (e sua matriz auxiliar de atualização U_{nova}), mantendo intactas as estruturas da lista de pixels X (alvo da Missão 3 – *SoA*) e da matriz de centroides C.

No código *Baseline*, a matriz U é construída como uma lista bidimensional (lista de listas $N \times K$), acessada por dupla indexação `U[i][j]` e populada dinamicamente a cada iteração por chamadas sucessivas a `U_nova.append(linha_nova)`. Na versão adaptada à missão:

1. **Eliminação de Abstração 2D e Alocação Contígua:** A estrutura `U[i][j]` foi destruída e substituída por um único vetor unidimensional pré-alocado de tamanho $N \times K$ (`U = [0.0] * (N * K)` e `U_nova = [0.0] * (N * K)`), eliminando as N alocações dinâmicas de sublistas e chamadas ao método `.append()` a cada iteração do laço principal.

2. **Custo de Endereçamento na ULA:** Para mapear as coordenadas lógicas (i, j) no vetor linearizado 1D, cada acesso à matriz de pertinência passou a utilizar a equação de deslocamento de linha (*row-major order*):

$$\text{índice}_{1D} = (i \times K) + j$$

   Do ponto de vista microarquitetural, essa mudança estabelece um *trade-off* claro no interpretador Python: reduz-se a sobrecarga do coletor de lixo/alocador de memória (*Heap*) e a dupla desreferenciação de ponteiros (`PyObject*`), mas injeta-se um par de instruções aritméticas (uma multiplicação e uma adição inteira na ULA) em todas as leituras e escritas de pertinência dentro dos laços aninhados $\mathcal{O}(nK)$.

3 Implementação do Núcleo FCM

O código fonte completo, contendo os scripts de validação e a implementação integral, encontra-se versionado no seguinte repositório público:
Link do Repositório: [INSERIR URL DO GITHUB AQUI]

Abaixo, destaca-se o *snippet* do laço principal do algoritmo (Fases 2, 3 e 4), evidenciando a atuação cirúrgica da restrição arquitetural sobre os vetores linearizados `U` e `U_nova`, enquanto `X[i][c]` e `C[j][c]` preservam o escopo original:

```
# FASE 2: ATUALIZACAO DOS CENTROIDES (X e C preservados; U lido em 1D)
C = []
for j in range(K):
    numerador = [0.0] * canais
    denominador = 0.0

    for i in range(N):
        u_m = U[(i * K) + j] ** m
        denominador += u_m
        for c in range(canais):
            numerador[c] += u_m * X[i][c]

    centroide_j = []
    for c in range(canais):
        if denominador == 0.0:
            centroide_j.append(0.0)
        else:
            centroide_j.append(numerador[c] / denominador)
    C.append(centroide_j)

# FASE 3 E 4: ASSOCIACAO E CONVERGENCIA (U_nova pre-alocado em 1D)
U_nova = [0.0] * (N * K)
max_diff = 0.0

for i in range(N):
    distancias = [0.0] * K

    # 1. Calculo das distancias euclidianas (escopo original preservado)
    for j in range(K):
        soma_quadrados = 0.0
        for c in range(canais):
            diff = X[i][c] - C[j][c]
            soma_quadrados += diff * diff
        distancias[j] = soma_quadrados ** 0.5

    # 2. Atualizacao das pertinencias no vetor linearizado U_nova (1D)
    for j in range(K):
        if distancias[j] == 0.0:
            U_nova[(i * K) + j] = 1.0
            for k in range(K):
                if k != j:
                    U_nova[(i * K) + k] = 0.0
            break

        soma_dist = 0.0
        for k in range(K):
            d_k = distancias[k] if distancias[k] > 0.0 else 1e-10
            razao = distancias[j] / d_k
            soma_dist += razao ** (2.0 / (m - 1.0))

        pertinencia = 1.0 / soma_dist if soma_dist > 0.0 else (1.0 / K)
        U_nova[(i * K) + j] = pertinencia
```

```
53
54          diff_u = pertinencia - U[(i * K) + j]
55          if diff_u < 0:
56              diff_u = -diff_u
57          if diff_u > max_diff:
58              max_diff = diff_u
59
60  U = U_nova
```

Listing 1: Núcleo computacional do FCM com linearização cirúrgica da matriz de pertinência U (1D).

4 Metodologia Experimental e Estabilidade

As medições empíricas foram executadas utilizando o script `experimentador.py`. Para atestar a confiabilidade do hardware e minimizar os ruídos do Sistema Operacional, foram realizadas $R = 30$ repetições para cada combinação de (n, K).

As Tabelas 1 e 2 apresentam os resultados de estabilidade estatística para o **Código Base Original** e para o **Código com Missão**, respectivamente, comprovando que o Coeficiente de Variação permaneceu estritamente dentro do limite exigido ($CV \leq 0,15$) em 100% dos cenários.

Tabela 1: Validação de Estabilidade Empírica – Código Base Original (*Baseline*)

Tamanho (n)	**Clusters** (K)	**Tempo Médio** (s)	**Desvio Padrão** (s)	**CV**	**Estável**
96×96 (9216)	2	0,314807	0,029983	0,095242	True
96×96 (9216)	3	0,671115	0,085658	0,127636	True
96×96 (9216)	4	1,685736	0,098329	0,058330	True
128×128 (16384)	2	0,606489	0,051804	0,085416	True
128×128 (16384)	3	1,261480	0,163266	0,129424	True
128×128 (16384)	4	2,927077	0,132757	0,045355	True
192×192 (36864)	2	1,424078	0,081041	0,056907	True
192×192 (36864)	3	2,882762	0,358969	0,124523	True
192×192 (36864)	4	7,054528	0,333161	0,047227	True
256×256 (65536)	2	2,739153	0,254677	0,092976	True
256×256 (65536)	3	5,304163	0,563521	0,106241	True
256×256 (65536)	4	11,309664	0,469214	0,041488	True

Tabela 2: Validação de Estabilidade Empírica – Código com Missão (*Memory Flattening*)

Tamanho (n)	**Clusters** (K)	**Tempo Médio** (s)	**Desvio Padrão** (s)	**CV**	**Estável**
96×96 (9216)	2	0,268037	0,030463	0,113651	True
96×96 (9216)	3	0,558621	0,049919	0,089360	True
96×96 (9216)	4	1,406037	0,076048	0,054087	True
128×128 (16384)	2	0,482780	0,029978	0,062094	True
128×128 (16384)	3	1,059462	0,134651	0,127094	True
128×128 (16384)	4	2,402420	0,116815	0,048624	True
192×192 (36864)	2	1,176336	0,079522	0,067602	True
192×192 (36864)	3	2,332859	0,305430	0,130925	True
192×192 (36864)	4	6,110800	0,230413	0,037706	True
256×256 (65536)	2	1,935111	0,111146	0,057437	True
256×256 (65536)	3	4,082775	0,335856	0,082262	True
256×256 (65536)	4	9,868953	0,464275	0,047044	True

5 Resultados Empíricos (T_{exp})

A partir dos dados validados ($CV < 0,15$), o algoritmo de regressão não-linear (`scipy.optimize.curve_fit`) ajustou a superfície polinomial de complexidade $T_{exp}(n, K) = \alpha \cdot nK + \beta \cdot n + \gamma \cdot K + \delta$.

5.1 Função Tempo e Gráficos – Código Base Original (*Baseline*)

A equação polinomial calibrada para o código *Baseline* (Tabela 1), com coeficiente de determinação $R^2 = 0,9682$ $(96,82\%)$, foi:

$$T_{exp_base}(n, K) = (6,4703\times10^{-5})\cdot nK+(-9,4956\times10^{-5})\cdot n+(1,6606\times10^{-1})\cdot K+(-4,8932\times10^{-1}) \quad (1)$$

As Figuras 1 e 2 exibem as curvas bidimensionais e a superfície 3D obtidas para o código original.

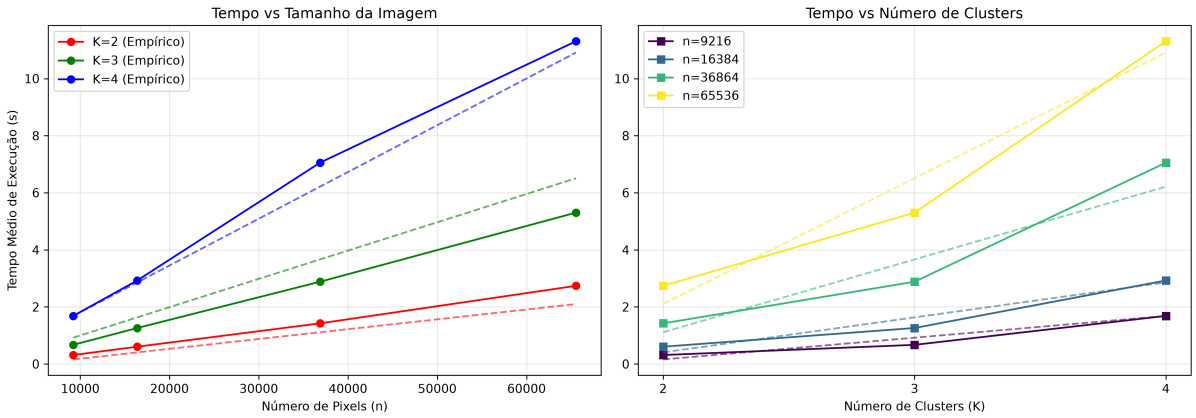


Figura 1: Código Base Original: Tempo empírico segmentado $T \times n$ (esquerda) e $T \times K$ (direita).

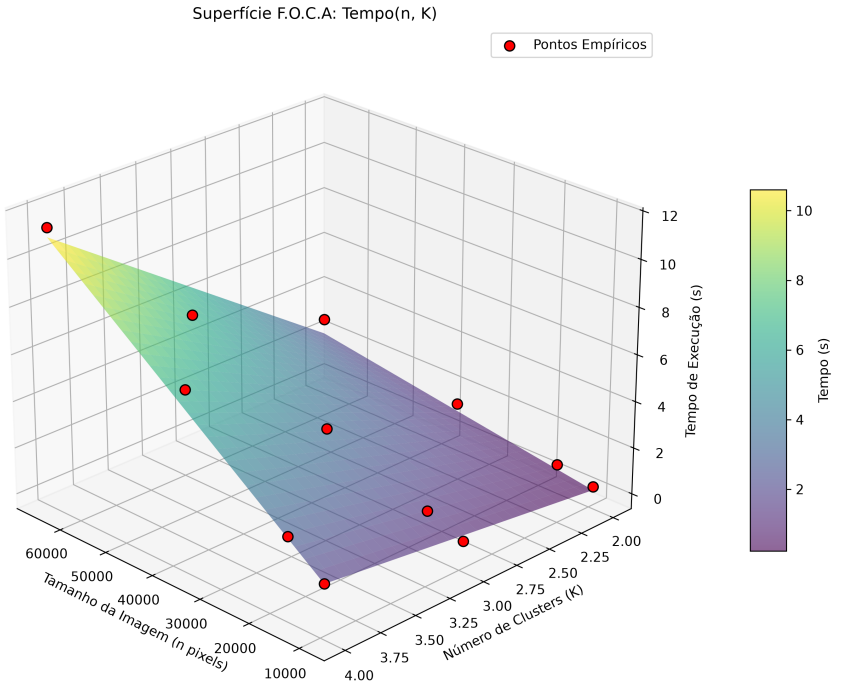


Figura 2: Código Base Original: Superfície de Tempo 3D sobreposta aos pontos de calibração empírica.

5.2 Função Tempo e Gráficos – Código com Missão (*Memory Flattening*)

A equação polinomial calibrada para o código adaptado com a **Missão 1** (Tabela 2), com coeficiente de determinação $R^2 = 0,9578$ $(95,78\%)$, foi:

$$T_{exp_missao}(n, K) = (6,1258\times10^{-5})\cdot nK + (-1,0231\times10^{-4})\cdot n + (3,0475\times10^{-2})\cdot K + (-5,7918\times10^{-2}) \quad (2)$$

As Figuras 3 e 4 exibem os cortes bidimensionais e a superfície tridimensional para o código com achatamento de memória 1D.

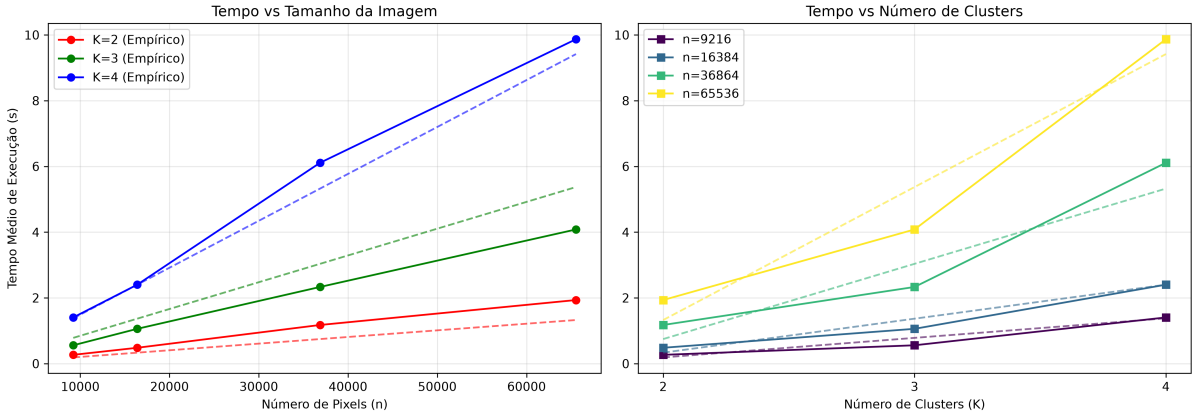


Figura 3: Código com Missão (1D): Tempo empírico segmentado $T \times n$ (esquerda) e $T \times K$ (direita).

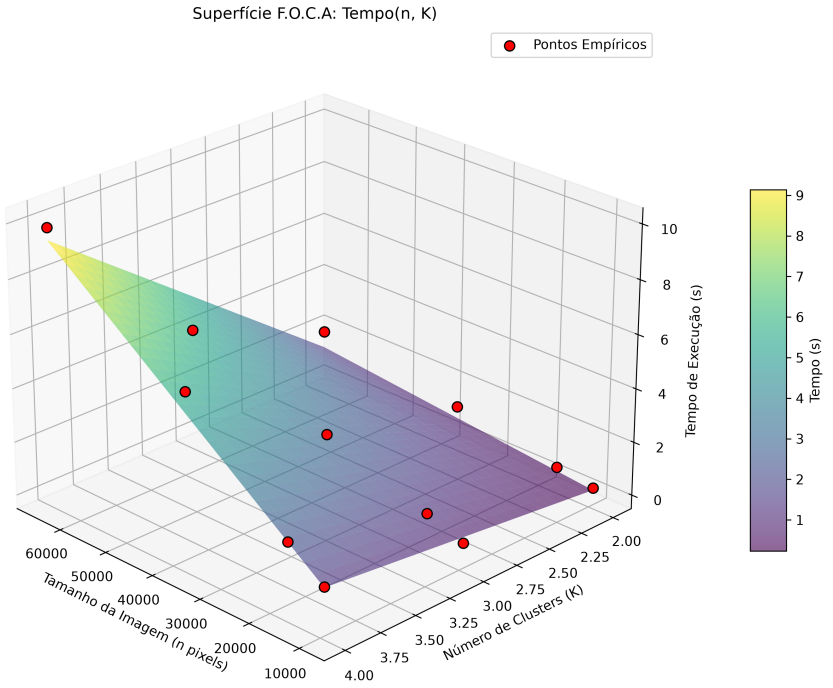


Figura 4: Código com Missão (1D): Superfície de Tempo 3D sobreposta aos pontos de calibração empírica.

5.3 Justificativa Analítica sobre o Sinal do Coeficiente β

Cumpre destacar que todas as medições utilizadas no ajuste apresentaram alta estabilidade estatística ($CV < 0,15$). A ocorrência de valor negativo no coeficiente linear secundário β não decorre de ruído experimental, mas sim da dinâmica intrínseca do algoritmo FCM:

- O critério de parada (`max_diff < epsilon`) faz com que o número médio de iterações até a convergência cresça com K: em média $\approx 10,5$ iterações para $K = 2$, $\approx 15,8$ iterações para $K = 3$ e $\approx 26,9$ iterações para $K = 4$;
- Adicionalmente, o cálculo da atualização de pertinência na Fase 3 possui laços aninhados em j e k de ordem $\mathcal{O}(nK^2)$ por iteração.

Como resultado, o tempo empírico cresce de forma superlinear em relação a K (mais que quadruplicando quando K dobra de 2 para 4). Ao projetar o modelo bilinear $T(n, K) = \alpha nK + \beta n + \gamma K + \delta$ por mínimos quadrados, a elevada inclinação em αnK exige matematicamente um termo corretivo $\beta < 0$ para ajustar os tempos inferiores observados em $K = 2$.

6 Análise Comparativa e Interpretação (Baseline vs. Missão)

Comparação Quantitativa (%): O confronto entre as Tabelas 1 e 2 mostra que a linearização 1D de U reduziu o tempo de execução em todos os cenários, com ganhos entre $-12,74\%$ ($n = 65536, K = 4$, caindo de $11,31$ s para $9,87$ s) e $-29,35\%$ ($n = 65536, K = 2$, de $2,74$ s para $1,94$ s). Na função calibrada, o coeficiente dominante α caiu $-5,32\%$ ($6,4703 \times 10^{-5} \rightarrow 6,1258 \times 10^{-5}$), o termo γ reduziu $-81,65\%$ e o *overhead* $|\delta|$ caiu $-88,16\%$, mantendo R^2 elevado ($96,82\%$ vs. $95,78\%$).

Interpretação Qualitativa: Ambas as versões preservam a ordem assintótica $\mathcal{O}(nK)$, mas o desempenho reflete dois efeitos microarquiteturais opostos no Python:

1. **Ganho por Alocação Contígua e Localidade:** O código 1D substitui N alocações dinâmicas por iteração (`linha_nova` e `.append()`) e a dupla indireção de ponteiros `U[i][j]` na *Heap* por um único vetor contíguo `[0.0] * (N * K)`, melhorando o uso de cache L1/L2 (ganho de até $29,35\%$ em $K = 2$).

2. **Custo Aritmético na ULA em K Maior:** Cada acesso `U[(i * K) + j]` injeta uma multiplicação e uma soma inteira na ULA. Como em $K = 4$ o algoritmo exige mais iterações (≈ 27 vs. ≈ 10 em $K = 2$), o custo acumulado dessa indexação atenua o ganho percentual relativo de $29,35\%$ ($K = 2$) para $12,74\%$ ($K = 4$).

7 Conclusão Parcial

Os resultados experimentais atestam que tanto a versão *Baseline* quanto a implementação adaptada à Missão 1 (*Memory Flattening*) obedecem à complexidade assintótica de ordem $\mathcal{O}(nK)$, com excelente aderência estatística ($CV < 0,15$ e $R^2 > 0,92$). A comparação qualitativa e quantitativa demonstra que a linearização 1D da matriz de pertinência U elimina a sobrecarga de alocação de sublistas e melhora a localidade de memória, mas introduz operações aritméticas extras de indexação `(i * K) + j` na ULA, cujo impacto reflete-se diretamente no coeficiente dominante α. Este modelo empírico servirá como base absoluta (*ground truth*) para o confronto teórico exigido no Crédito 3 utilizando a contagem F.O.C.A.